

Initial Thoughts on the Requirements Register

Working feedback for Timothy

My first reaction is very positive. The register feels like the right next step because it gives us one place to see what has been raised, what has been designed, what is partially addressed, and what is actually implemented. More importantly, it creates a visible record of how the thinking is moving into the framework instead of allowing good ideas to disappear across calls, emails, working notes, and PDFs.

What I like most is that this starts to give the collaboration memory - not just a list of comments, but a traceable path from concern to design to implementation.

What feels strong already

- The top-level counts make the state of the work easy to understand at a glance. I can immediately see what is implemented, designed, partially addressed, and still open.
- Tracing each requirement back to its source is valuable. It preserves context and makes it easier to understand why a requirement exists in the first place.
- The categories are useful because they show that the feedback is not all the same kind of work. Website/UX, readiness, data model, executive view, and people/org taxonomy issues should not all be treated as one bucket.
- The draft banner is also helpful. It makes clear that this is still a working tracker and not yet part of the formal framework.

Where I think we can make it more useful

1. Define the statuses more tightly

Open / Partially Addressed / Designed / Implemented makes sense, but I would define exactly what each status means. “Designed” is the one I would be most careful with. Does it mean the concept has been drafted, that both of us reviewed it, that we agreed to it, or that it is ready to be built? Those are different stages.

2. Add ownership and next action

For every open or partial item, I would like to know who carries the next move and what that move is. That could simply be Tim, Gigi, Joint, or Technical Implementation. Without that, the register tells us where something stands but not how it moves forward.

3. Add a sense of priority

Not all requirements should carry the same weight. A typography adjustment and a Human Readiness gating issue are both legitimate requirements, but they do not have the same impact on

the integrity of the methodology. I would use something simple such as Framework-Critical / Important / Enhancement, or High / Medium / Low.

4. Make “done” explicit

I would add acceptance criteria, even if it is only one sentence. That gives us a shared definition of what has to be true before something can be called implemented. It also prevents a requirement from being marked complete when the original concern was only partly solved.

An example of how I would use it

Requirement	Human Accountability Anchor
Status	Partially Addressed
Owner	Joint
Priority	Framework-Critical
Next Action	Define when an activity requires an explicit accountability marker and test it against the three prototype roles.
Acceptance Criteria	We can identify the human who remains accountable, their override/escalation rights, and how that accountability is represented in the framework.

One distinction I think matters

Designed is not the same as reviewed or agreed. I would consider a review step between those states so there is a visible difference between “a solution has been designed” and “we have both reviewed and accepted that solution.” That could be a formal status such as Pending Review, or it could be handled through an approval field.

A few process questions I would keep open

- When does a requirement move from this register into the actual framework?
- Who can change a requirement or its status?
- What evidence is needed before something moves from Designed to Implemented?
- If we disagree that a requirement has been fully addressed, where is that disagreement recorded?
- Should the register eventually capture input from both of us, pilots, practitioners, and clients - not only my initial requirements?

How I see this evolving

I would not want this to remain only a tracker of my comments. In the short term, that is a useful way to make sure my feedback is not lost. Longer term, I think it can become the requirements-governance system for the methodology itself - capturing requirements raised by either of us,

findings from pilots, practitioner feedback, validation issues, and eventually client-driven requirements.

If we get the definitions, ownership, next action, priority, and acceptance criteria right, this can become the backbone for how we manage the methodology as it grows.

My overall reaction

I think this is a strong move and exactly the kind of mechanism we needed. It makes the work visible, creates accountability around follow-through, and gives us a record of how ideas move from reflection into the architecture. My suggestions are mostly about making it a little more operational so that we can look at any row and immediately know not only where it stands, but what happens next and what “complete” really means.

Gigi